Speech Module - Technical Documentation

Mateusz Block, Lucjan Butko, Adam Macko

March 2026

Contents

1 Project Overview and Purpose

The main goal of the speech module, developed as part of the "humanoid-head" project, is to enable the robot to conduct natural and meaningful conversations with humans. Our team's task is to ensure that the machine not only reacts to rigid commands but can also respond fluently and logically to questions, mimicking a real conversation.

The assistant was designed with the students of the Gdańsk University of Technology in mind. Its practical applications within this interaction include:

- Providing precise navigation directions, making it easier to reach specific rooms in university buildings (such as EA or NE).
- Providing up-to-date information about weather conditions.
- Engaging in casual conversations on general topics, using artificial intelligence to generate natural-sounding responses.

2 System Architecture

The system is designed as a continuous loop that runs until it is stopped manually. It uses a pipeline structure with three main steps:

- **Speech-to-Text (STT):** The system uses a microphone to listen to the user. It records the sound and uses the Whisper AI model (the "small" version) to change the user's voice into text.
- **Intent Detection and Processing (LLM):** After getting the text, the system checks what the user wants using the `IntentDetector`.
  - If the user asks about the weather ("POGODA") or university classes ("PG"), the system uses specific logic to find this data.
  - For all other questions, it sends the text to a Large Language Model. The project uses the Ollama framework with the "gemma3:4b" model, connected through the LangChain library. The model gets a special prompt with a database of rooms and weather data to know how to answer correctly.
- **Text-to-Speech (TTS):** When the text answer is ready, the system uses the `pyttsx3` library to read it out loud.

3 Environment and Dependencies

3.1 Required Libraries

The project relies on a variety of Python libraries for audio processing, web scraping, system integration, and data manipulation. The key dependencies are described below:

- **BeautifulSoup4** (`beautifulsoup4`): Used for parsing HTML documents. It is utilized to scrape information and extract staff details from the university's websites.
- **LangChain** (`langchain-core`, `langchain-ollama`): Manages the integration with local Large Language Models via Ollama and formats chat prompt templates.
- **PyAudio** (`pyaudio`): Provides Python bindings for PortAudio, essential for capturing live audio streams from the microphone.

- **Python Weather** (`python-weather`): An asynchronous library used to fetch current weather data and conditions.
- **pyttsx3** (`pyttsx3`): An offline Text-to-Speech (TTS) conversion library that allows the assistant to vocalize its responses to the user.
- **RapidFuzz** (`rapidfuzz`): A fast string matching library used for fuzzy search. It helps in matching spoken teacher names against the database, accommodating slight mispronunciations or transcription errors.
- **Requests** (`requests`): An HTTP library utilized to send requests to external university web pages for data scraping purposes.
- **SpeechRecognition** (`speechrecognition`): A wrapper library that facilitates capturing microphone input, detecting speech, and automatically adjusting for ambient noise levels.

3.2 AI Models

The system implements the following artificial intelligence models and their associated frameworks to process user input, recognize entities, and generate appropriate responses:

- **Whisper ("small")**: A Speech-to-Text (STT) model developed by OpenAI (managed via the `openai-whisper` library), used to transcribe the audio captured from the user's microphone into text.
- **Gemma 3 (`gemma3:4b`)**: A Large Language Model (LLM) served via Ollama. It is responsible for generating conversational responses for general queries when no specific predefined intent is detected.
- **GLiNER (`urchade/gliner_multi-v2.1`)**: A Named Entity Recognition (NER) model (implemented via the `gliner` library) utilized to extract specific information from the transcribed text. It identifies entities such as room codes and person names (using `room code` and `person` labels) to assist in university navigation queries.
- **spaCy (`pl_core_news_sm`)**: A pre-trained Polish language model operating within the `spacy` NLP library. It performs word lemmatization and tokenization to quickly and accurately match user queries against predefined intent keywords.

4 Core Modules Implementation

4.1 NLP Pipeline

The Natural Language Processing (NLP) pipeline acts as the central coordinator of the humanoid head's interaction system. Implemented primarily within the `NlpModel` class, it manages the continuous loop of listening, understanding, and responding to the user. The pipeline is composed of three main stages:

1. **Speech-to-Text (STT):** The pipeline continuously captures ambient audio via the microphone using the `SpeechRecognition` library, automatically adjusting for background noise. Once speech is detected, the audio is processed by the Whisper STT model to generate an accurate text transcription.
2. **Intent Routing & Processing:** The transcribed text is immediately forwarded to the Intent Detection module. This module classifies the user's query into specific operational branches:

- If a **Weather Intent** is detected, the system bypasses the LLM and directly queries the weather integration module to retrieve the current conditions.
- If a **PG (Politechnika Gdańska) Intent** is detected, the text is preprocessed and analyzed by the GLiNER model to extract entities such as room codes or personnel names, which are then passed to the university database modules.
- If **No Specific Intent** (General) is detected, the transcribed text is fed into the locally hosted Large Language Model (Gemma 3) via LangChain to generate a natural, conversational response.

3. **Text-to-Speech (TTS):** Regardless of which branch processes the query, the resulting answer is sent to the TTS module (`pyttsx3`) to be synthesized into audible speech, completing the interaction cycle.

To better illustrate the data flow within the `NlpModel`, Figure 1 presents the architectural diagram of the pipeline.

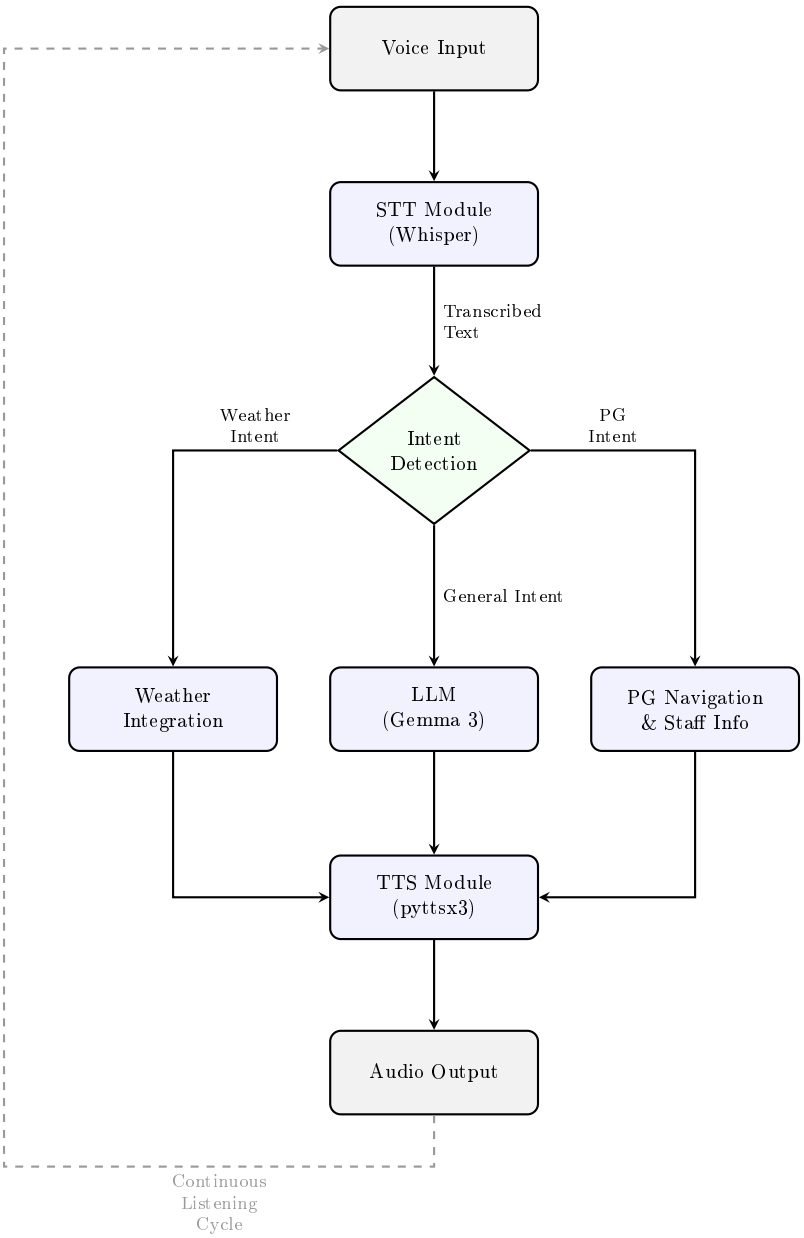

Figure 1: NLP Pipeline Data Flow.

4.2 Intent Detection

The intent detection module is responsible for classifying the user's transcribed speech into one of the predefined categories, which determines how the query is routed within the NLP pipeline. This logic is encapsulated within the `IntentDetector` class.

Initialization and Keyword Sets

The module uses the `spaCy` library along with the Polish language model (`pl_core_news_sm`) to perform advanced text analysis. During initialization, it defines two primary sets of base words used

for classification:

- **PG Keywords:** Terms related to the university, such as "sala", "wykład", "harmonogram", "profesor" and "dojść".
- **Weather Keywords:** Terms related to meteorological conditions, such as "pogoda", "temperatura", "deszcz" and "wiatr".

Operating Principle

The core classification is handled by the `detect_intent()` method, which processes the input through the following steps:

1. **Text Preprocessing:** The raw input string is converted to lowercase and passed through the spaCy processing pipeline.
2. **Lemmatization and Filtering:** The module iterates over the generated tokens, actively filtering out punctuation marks (`is_punct`) and common Polish stop words (`is_stop`). It extracts the base form (lemma) of each remaining token and stores them in a set for optimized searching.
3. **Set Intersection and Classification:** The system uses logical set intersections to check for overlaps between the extracted lemmas and the predefined keyword sets.
   - If an intersection with the university keywords is found, it returns the `"PG"` intent.
   - If an intersection with the weather keywords is found, it returns the `"POGODA"` intent.
   - If there are no matches, the function returns `"OGOLNA"` intent (general intent), signaling the pipeline to process the query using the large language model.

4.3 Weather Integration

4.4 Rooms Database and Navigation

To provide reliable and precise navigation without relying on the unpredictable nature of Large Language Models (which are prone to hallucinations), the system utilizes a deterministic database of room locations.

The navigation system is built upon three main components:

- `create_rooms_database.py`**:** A Python script responsible for generating the navigation rules. The architectural layouts of the university buildings (EA and NE buildings) were thoroughly analyzed using the official interactive map (`https://campus.pg.edu.pl/`). Based on this spatial data, custom algorithms were devised to group rooms into logical clusters. The script generates custom, natural-language directions in Polish (e.g., "Pojedź windą lub pójdź schodami na piętro 3...").
- `room_directions.json`**:** The output file generated by the aforementioned script. It serves as a static, lightweight JSON database mapping building codes and room numbers to their respective floors and exact walking directions.
- `find_room.py`**:** A search algorithm designed to assist the Large Language Model. When the pipeline extracts a specific room code from the user's speech, this script queries the JSON database and retrieves the exact navigational string. This data is then injected into the system to ground the assistant's response in factual, data.

4.5 Staff Information

Similarly to the spatial navigation module, the assistant is equipped to provide information regarding the university's faculty members, including their academic titles, affiliated departments, and exact office locations.

The staff database and retrieval logic are managed by the following components:

- `teachers_info.py`**:** A web scraping script designed to systematically extract faculty data from the official Gdańsk University of Technology department websites. It parses HTML documents to capture names, titles, and room numbers. However, due to inconsistencies in how information is formatted across different profile pages, the automated scraping was not entirely sufficient. Consequently, a manual verification phase was mandatory to review and correct the scraped data, ensuring the highest accuracy of the database.
- `teachers_info.json`**:** The compiled database resulting from the scraping and manual review. It stores structured profiles for each staff member, mapping their full name to their academic title, assigned building, room number, and department. In cases where specific information (such as an office assignment) is unavailable or unlisted, the corresponding field is explicitly set to `"Brak"`. This specific flag prevents the system from generating false or hallucinated locations.
- `find_teacher.py`**:** A retrieval algorithm conceptually similar to `find_room.py`, but tailored specifically for querying the staff database. This allows the assistant to successfully identify the correct staff member even if the recognized name is slightly distorted.

5 Prompt Engineering

5.1 System Prompt

The system prompt establishes the assistant as a helpful guide for students of the Gdańsk University of Technology.

If the intent module detects a general conversational intent rather than a specific command, the system uses this prompt to guide the Large Language Model. The user's transcribed query is simply injected into the `{question}` placeholder, allowing the AI to generate a natural, contextual response.

5.2 Deterministic Retrieval and Tool Calling

During the initial development phases, the implementation of a Retrieval-Augmented Generation (RAG) architecture was considered. Ultimately, the RAG approach was abandoned in favor of a deterministic "Tool Calling" architecture.

When the system detects a specific, university-related intent, it bypasses the LLM's generative capabilities. Instead, the text is processed by the **GLiNER** model (for Named Entity Recognition) to extract key parameters, which then triggers dedicated Python scripts to fetch hardcoded, verified data. The two primary retrieval tools used in this process are:

- `find_room.py` **(Exact Match Retrieval):** This script is executed when the GLiNER model extracts a `room code` entity. It takes a formatted input string, splits it into building and room components using a comma delimiter, and performs a strict exact-match search within the `room_directions.json` database. If a match is found, it directly returns the precise, step-by-step navigation instructions.
- `find_teacher.py` **(Fuzzy Match Retrieval):** This script is invoked when a `person` entity is detected by GLiNER. Because Speech-to-Text models often mistranscribe proper nouns and surnames, an exact match approach would frequently fail. To solve this, the script utilizes the `rapidfuzz` library to calculate a string similarity score against the `teachers_info.json`

database. If the match score meets or exceeds the defined threshold of 50%, the system assumes a successful identification and retrieves the professor's assigned office location.

By employing these retrieval scripts as external tools, the system guarantees factual accuracy in its navigational responses and significantly reduces latency, completely mitigating the risk of LLM hallucinations.